



SAP Application Development (S/4 HANA)

Modularization

Content

This chapter is about subdividing program parts in several independent function modules. You will learn how to create and use function modules to reduce the complexity of your main program.

Prerequisites

You should be familiar with basic ABAP development and have experience in navigating with SAP GUI and Eclipse. Furthermore, you should have completed the chapter *Introduction to SAP Programming*.

Motivation

Nowadays, it's hard to fulfill both application requirements and good conditions to enable easy source code maintenance. That's why developers should try to reuse existing program parts as often as possible and follow the paradigm "divide-and-conquer" by making use of modularization. This chapter is an excellent introduction to start modularization by creating and using function modules.

Lecture notes

Fundamental knowledge about ABAP development is a prerequisite for the students. Students can go on with their account from the previous chapters.

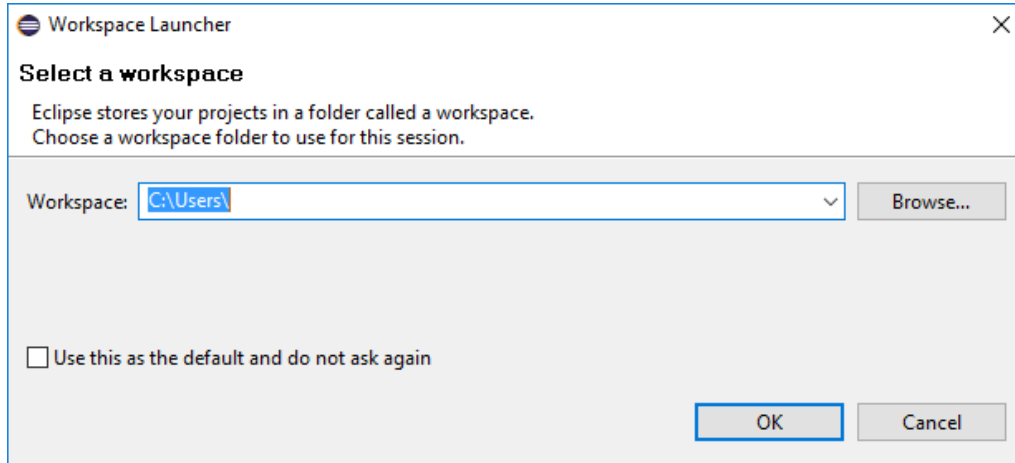
- **Product:** All
- **Level:** Advanced
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Login into the SAP system within Eclipse

Short description: Use Eclipse to login into the SAP system with your username and password

Start Eclipse and select the local workspace where your ABAP project is stored.

Login



Please refer to the local workspace of **Module “Basic Concepts”** where you learned how to create first programs in ABAP. If you haven't finished the exercise from **Module “Basic Concepts”**, please perform task 2 and 3 of the exercise before you continue with this one.

Task 2: Code outsourcing – making use of function modules

Short description: Use a function module to separate a certain method from the main program

You already learned how to use the main program to perform certain calculations. With this exercise you get to know how to organize your program more effectively by outsourcing as much as possible to function modules.

In this task we want to separate the calculation from the main program and want to use a function module instead. Therefore, create a new function group called 'Z_\$\$_###_CAR_FUNCTIONGROUP'. This can be done easily by performing a right click onto your Source Library:



New ABAP function group

ABAP function group
Create an ABAP function group

Project: * ABAP_000 Browse...

Package: * Z_000 Browse...

☐ Add to favorite packages

Name: * Z_000_CAR_FUNCTIONGROUP

Description: * A new function group

Original language: EN

Please refer to a previous transport request or create an own one if you want to separate this work from the other ones you did before. Afterwards, perform a right-click onto your new Function Group and select

New • Function Module

Please create a new function module which is named **Z_\$\$\$_CAR_FM_CALCULATION**.

New ABAP function module

ABAP function module
Create an ABAP function module

Project: * ABAP_000 Browse...

Name: * Z_000_CAR_FM_CALCULATION

Description: * Function module for calculation

Original language: EN

Function group: * Z_000_CAR_FUNCTIONGROUP Browse...

In the source code of your new Function Module perform a right-click and select:

Open With • SAP GUI

Switch to the 'Import' tab and activate the edit-mode. In our scenario we want to import three parameters: **IM_TRIP_F**, **IM_TRIP_M** and **IM_COUNTRY**. Besides, you have to define the parameters data type and activate the checkbox for passing their values.

Import

Function module Z_000_CAR_FM_CALCULATION Inactive (Revised)

Attributes **Import** Export Changing Tables Exceptions Source code

Parameter Name	Typi...	Associated Type	Default value	Op...	Pa...
IM_TRIP_F	TYPE	I		☐	☒
IM_TRIP_M	TYPE	I		☐	☒
IM_COUNTRY	TYPE	C		☐	☒

Now switch to the '**Export**' tab. Here you have to define all variables to be exported by the function module. Please define the variables **EX_RESULT** and **EX_TEXT** to be exported.

Export

Function module		Z_000_CAR_FM_CALCULATION	Inactive (Revised)			
Attributes	Import	Export	Changing	Tables	Exceptions	Source code
						
Parameter Name	Typing	Associated Type	Pass Val...	Short text		
EX_RESULT	TYPE	F	☒			
EX_TXT	TYPE	STRING	☒			

Save and activate your changes now. Afterwards, close the integrated SAP GUI. Refresh your source code in Eclipse. The interface definition has already been created by the system. If you change the interface definition here, it will not affect the importing and exporting parameters as this is just text without any further semantic behind. In the source code of your function module you implement the new '**CASE**' instruction whereas you have to replace the old parameter names (e.g. trip_f) with the new ones (IM_TRIP_F). Your new source code should look similar to this:

```

1 FUNCTION Z_CAR_MR_FM_CALCULATION
2   IMPORTING
3     IM_TRIP_F TYPE I
4     IM_TRIP_M TYPE I
5     IM_COUNTRY TYPE C
6   EXPORTING
7     VALUE(EX_RESULT) TYPE F
8     VALUE(EX_TEXT) TYPE STRING.
9
10
11
12 CASE im_country.
13   WHEN 'G'.
14     ex_result = im_trip_f / ( im_trip_m / 100 ).
15     ex_text = 'l/100km'.
16   WHEN 'U'.
17     ex_result = im_trip_m / im_trip_f.
18     ex_text = 'miles per gallon'.
19   ENDCASE.
20
21 ENDFUNCTION.

```

Do not forget to save, check and activate the function module/group. You may test the new function module directly.

In this step we want to replace the case instruction in your program by a function call. Obviously the function to be called is your new created function **Z_\$\$\$ CAR_FM_CALCULATION**. Therefore, switch back to your program **Z_\$\$\$ CAR**. Add a function call by entering 'CALL FUNCTION' and use the auto completion feature by pressing **Ctrl + Space**, select your function module and press **Shift + Enter** in order to let the system automatically insert the new function call and a kind of skeleton. You only have to assign the variables to the importing and exporting parameters.

Call Function**Hint:**

When you call a function within your program the exporting parameters always describe parameters, which are exported to the external function module. The importing parameters are parameters, which are imported to the current program.

Importing vs. Exporting

Complete the source code similar to the following screenshot:

```

1 REPORT z_000_car.
2
3 TYPES: BEGIN OF str_car,
4     brand(40)      TYPE c,
5     model(40)      TYPE c,
6     mileage        TYPE i,
7     fueltank_cap   TYPE i,
8     fuel_efficiency(6) TYPE c,
9 END OF str_car.
10
11 PARAMETERS: trip_f TYPE i, trip_m TYPE i, country TYPE c.
12
13 DATA result TYPE f.
14 DATA text TYPE string.
15 DATA coolcar TYPE str_car.
16
17 coolcar-brand = 'Mercedes'.
18 coolcar-model = 'SLR'.
19 coolcar-mileage = '10254'.
20 coolcar-fueltank_cap = '70'.
21
22 CALL FUNCTION 'Z_000_CAR_FM_CALCULATION'
23     EXPORTING
24         im_trip_f = trip_f
25         im_trip_m = trip_m
26         im_country = country
27     IMPORTING
28         ex_result = result
29         ex_text = text
30     .
31
32 WRITE result TO coolcar-fuel_efficiency EXPONENT 0.
33 WRITE: / 'Brand: ', coolcar-brand, / 'Model: ', coolcar-model, / 'Mileage: ',
34         coolcar-mileage, / 'Fuel-Tank Capacity: ', coolcar-fueltank_cap, / 'Result: ',
35         coolcar-fuel_efficiency, text.

```

Save, check and activate your program. Now you can test the program with your new function call.

Activate everything!

Challenge:

Short description: Extend your program with a subroutine to convert miles in kilometers in order to provide the information of fuel efficiency also within European measurement units.